CRM Platform — Phase 1 Task List

Multi-tenant CRM · Next.js + Prisma + PostgreSQL + Docker · Full-stack team + AI agents team

Schema design rules — lock these in before anyone writes a model

- *No enums anywhere in the schema. Every classification (role, lead status, lead source, call type, call outcome, etc.) is its own lookup table with id, name, sortOrder and isActive.*
- *Lookup tables take a nullable companyId — null means a global default, set means a company-specific override. This is how each tenant customizes their own pipeline later.*
- *Frontend form fields are data, not code. FieldDefinition + FieldValue tables let each company add custom fields to Leads/Clients, and the React form renders off that table.*
- *Hierarchy is Organization → Company → Employee, with Employee.managerId as a self-relation for the org chart. Platform admin is a role, not a table — there is only one platform.*

Task 1 · Both teams

Do this first, individually

- Clone the repo and install Docker Desktop.
- Run **`docker compose up`** and confirm the Next.js app and Postgres containers report healthy.
- Run **`npx prisma migrate dev`** against the local database.
- Open Prisma Studio or pgAdmin and confirm you can connect and see tables.
- Report working / not working in the team channel before any schema work begins — everyone branches off the same migration history, so a broken environment blocks the group.

Full-stack developer team

Tasks 1–9, in order

1 Design the tenancy schema together, before splitting up

- Hierarchy: Organization (created by platform admin) → Company (created by an org) → Employee (belongs to a company).
- Employee.managerId is a self-relation back to Employee — this is how the org chart and reporting lines work.
- Platform admin is a role on a user, not its own table.

2 Build the lookup-table pattern and reuse it everywhere (no enums)

- Generic shape: id, name, sortOrder, isActive, companyId (nullable).
- Apply it to: Role, LeadStatus, LeadSource, CallType, CallOutcome.
- Get this pattern right first — every other table in the app references one of these.

3 Build RBAC off the Role table

- RolePermission join table maps roles to permission strings, e.g. lead.create, employee.invite.
- Middleware checks permissions against this table instead of hardcoded role checks in code.

4 Build the frontend-config-in-schema layer

- FieldDefinition: entityType, fieldKey, label, fieldType, options (JSON), isRequired, companyId (nullable).
- FieldValue: entityId, fieldDefinitionId, value.
- Lead/Client forms and table columns render dynamically off FieldDefinition instead of being hardcoded in React.

5 Build Organization / Company / Employee CRUD

- Platform admin creates orgs, org admin creates companies, company admin invites employees and sets their manager.

6 Build lead import and CRUD

- Excel (xlsx) and CSV (papaparse) upload with a column-mapping step.
- Validation and duplicate detection by phone/email, scoped to a company.
- Reference LeadStatus and LeadSource by foreign key, never by string.

7 Build status transitions and the LeadStatusHistory audit table

- Columns: leadId, fromStatusId, toStatusId, changedBy, changedAt, note.

8 Build manual call logging and lead-to-client conversion

- Employee call log UI.
- Auto-convert a lead to a Client record when it hits the company's “won” LeadStatus row.

9 Seed data and a tenant-isolation test

- Create two orgs, two companies each, and confirm a company admin in org A can never query org B's leads or employees.
- This is the test that catches multi-tenancy bugs before they become security bugs.

AI agents developer team — Tasks 1–8, in order

1 Same Task 1 above — Docker setup

2 Sit in on the full-stack team's schema session and add these to the same migration

- Call: leadId, callTypeId (→ CallType), employeeId (nullable), startedAt, endedAt, recordingUrl.
- Transcript: callId, rawText, language.
- Assessment: callId, shortDescription, outcomeId (→ CallOutcome), generatedBy.
- ScheduledAction: leadId, nextCallAt, assignedToType (employee or AI agent), assignedToId (nullable).
- One shared schema avoids a painful merge later — do not branch a separate migration.

3 Research and pick the voice + speech-to-text stack

- Compare a calling API (Twilio, Vonage) against an all-in-one voice-agent platform (Vapi, Bland, Retell).
- Decide before writing any integration code.

4 Build a standalone calling prototype

- Trigger one outbound call, capture the transcript, write it into the Transcript table.
- No UI needed — a script hitting the shared Postgres instance, runnable in parallel with the full-stack team's CRUD work.

5 Build Agent 4 — transcript to assessment

- Design the prompt that turns a raw transcript into a short description and an outcome classification.
- Write results into the Assessment table.

6 Build Agent 5 — next-action scheduler

- Reads the Assessment and decides ScheduledAction: next call time, and employee vs AI agent.

7 Wrap Agents 3–5 in an async worker

- BullMQ + Redis, or a small separate service, so calls process without blocking the main app.
- Webhook back into the CRM so a completed call updates the lead's status and timeline in real time.

8 Joint integration test with the full-stack team

- Trigger a call from the actual CRM UI on a real lead.
- Confirm it flows through Call → Transcript → Assessment → ScheduledAction and shows up on the lead's timeline.